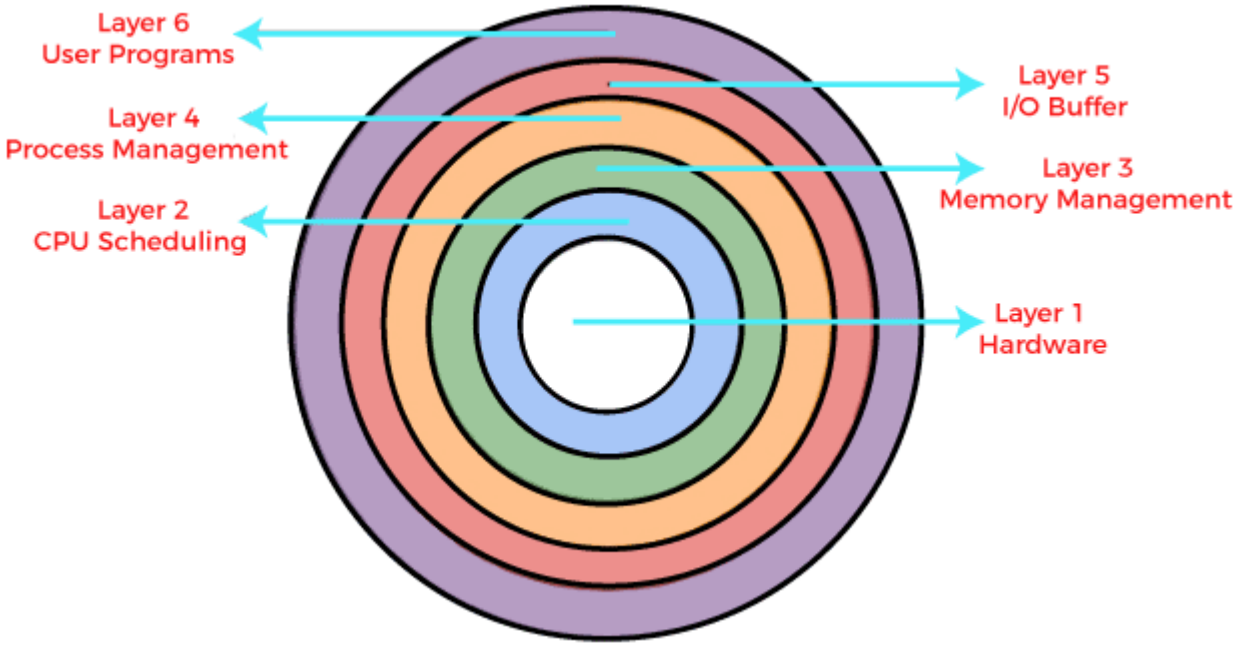
İşletim Sistemleri
(Operating Systems)



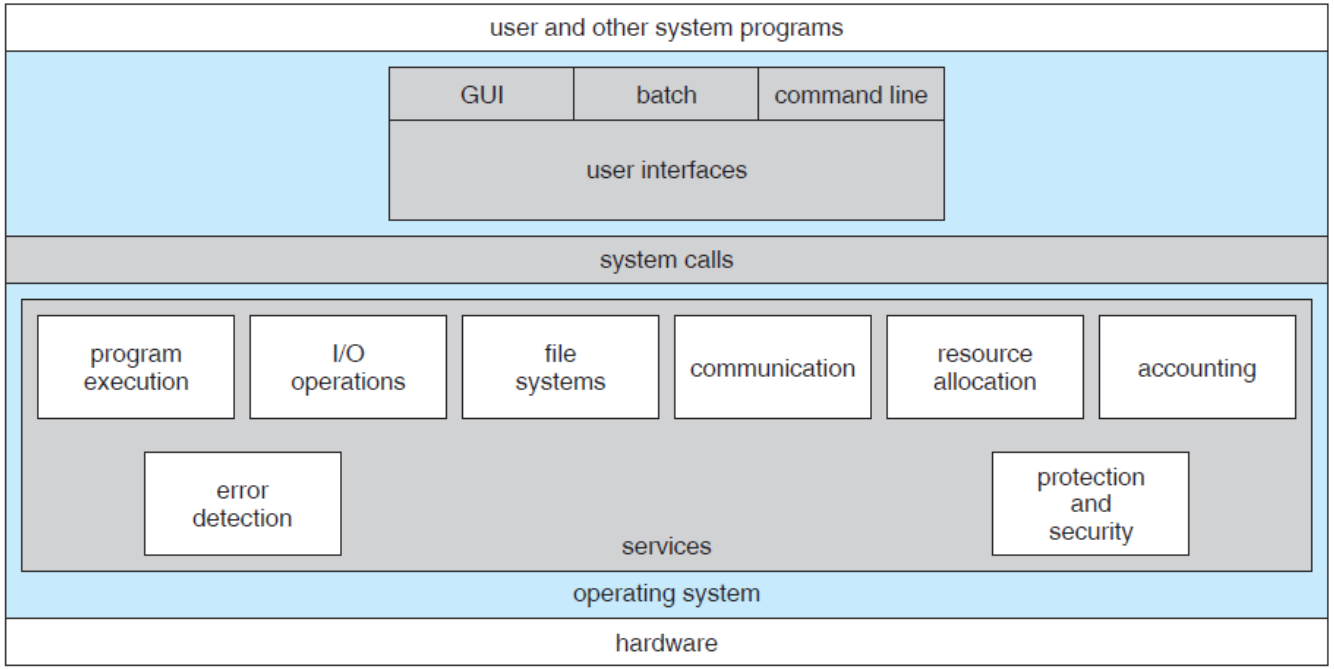
İşletim Sistemleri Yapıları
(Operating-System Structures)

İşletim Sistemi Hizmetleri (Operating System Services)

- İşletim sistemi, programlara ve bu programların kullanıcılarına belirli hizmetler sağlar.
- Kullanıcılara programları uygun bir şekilde yürütmeleri için hizmetler sunar.

- İşletim sistemi tarafından sağlanan yaygın hizmetler:

- Kullanıcı arayüzü
- Program yürütme
- Giriş/Çıkış işlemleri
- Dosya Sistemi
- İletişim
- Kaynak Tahsisi
- Kullanıcı işlemleri
- Hata Tespiti
- Koruma ve Güvenlik



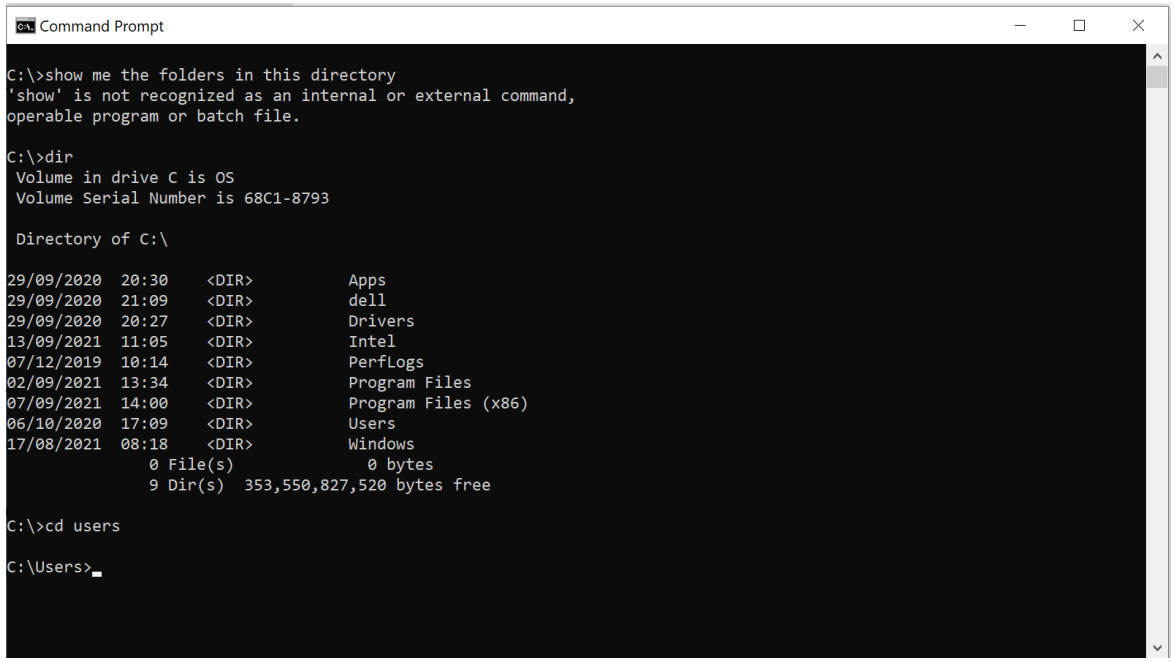
Kullanıcı arayüzü (User interface)

- Hemen hemen tüm işletim sistemlerinin bir kullanıcı arayüzü vardır. Bu arayüz birkaç şekilde olabilir.
- **Komut satırı arabirimi (command line interface - CLI)**
- **Toplu iş arabirimi (batch interface)**
- **Grafik kullanıcı arayüzü (graphical user interface - GUI)**

- Bazı sistemler bu arayüz varyasyonların ikisini veya üçünü sağlar.

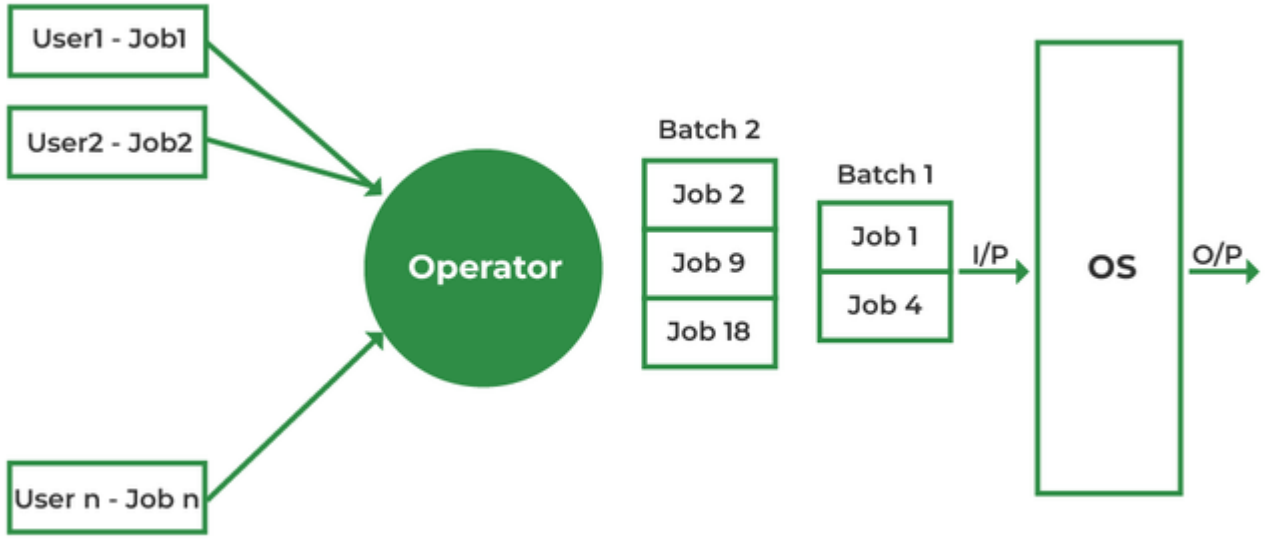
Komut Satırı Arabirimi (Command Line Interface - CLI)

- Kullanıcının belirli metin komutları yazarak sistemle etkileşime girmesine izin verir.
- Bu komutlar ayrıca bir terminal emulator veya uzak kabuk istemcisi kullanılarak da girilebilir.
- Grafik olmadığı için düşük miktarda ana bellek ve ikincil depolama gerektirir.
- Deneyimli kullanıcılar ve sistem yöneticileri de, genellikle bir grafik kullanıcı arabirimine uygulanan kısıtlamalardan çok bilgisayar sistemi üzerinde daha fazla kontrole izin verdiği için bir CLI kullanmayı tercih eder.



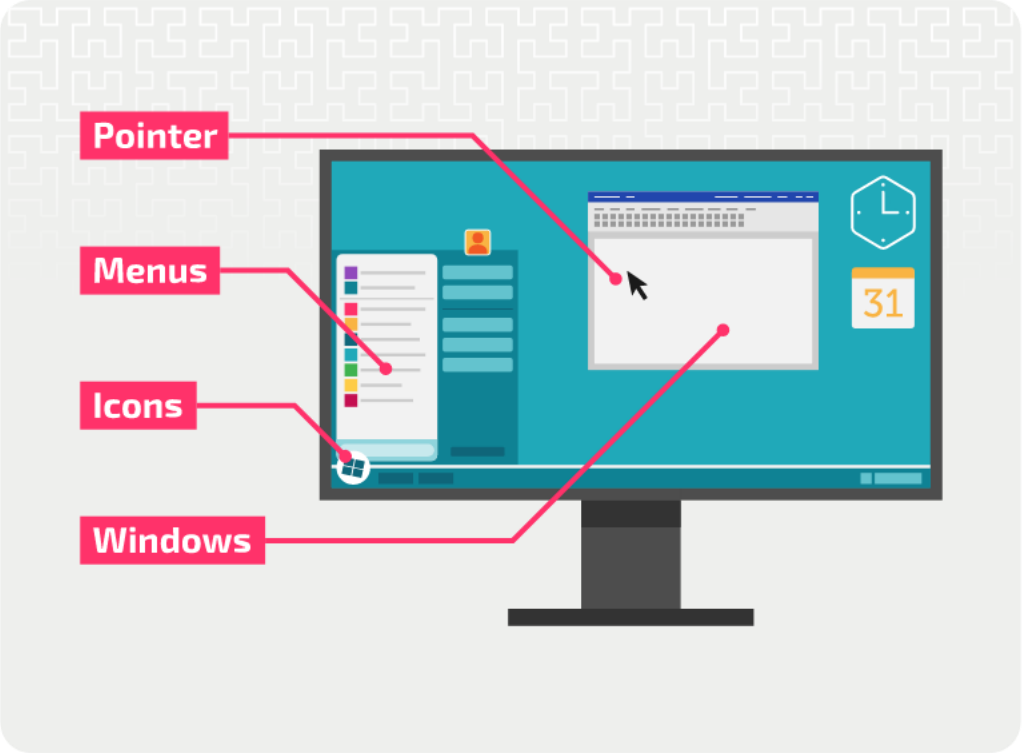
Toplu iş arabirimi (batch interface)

- Komutları kontrol etmek için komutların ve direktiflerin dosyalara girildiği ve bu dosyaların yürütüldüğü ortam.



Grafik kullanıcı arayüzü (Graphical User Interface - GUI)

- Masaüstü bilgisayarlardan akıllı telefonlara kadar çok çeşitli bilgisayar sistemlerinde bulunan en yaygın kullanıcı arayüzüdür.
- Tipik olarak bir GUI, Pencereler, Simgeler, Menüler ve İşaretçiden (fare tıklamaları veya ekran basmaları) oluşur.
- Görsel ve sezgisel olarak tasarlandıkları için kullanıcı dostu bir arayüz olarak kabul edilirler.
- Bunun nedeni, kullanıcının bir komut satırı arabirimi için gereken belirli komutları öğrenmesine gerek kalmadan simgeler ve menülerle (klavye, fare veya dokunmatik ekran gibi giriş aygıtlarını kullanarak) etkileşimde bulunabilmesidir.
- Diğer kullanıcı arabirimi türleriyle karşılaştırıldığında, bir GUI grafik içerdiğinden, daha yüksek bir ana bellek ve ikincil depolama kapasitesine gereksinim vardır.



Program yürütme (Program execution)

- İşletim sistemi, hem kullanıcı hem de sistem programlarının sorunsuz yürütülmesinden sorumludur.

- Bir programı belleğe yükler.
- Bellekteki process'in yürütülmesini yönetir.
- Process senkronizasyonu için bir mekanizma sağlar.
- Process iletişimi için bir mekanizma sağlar.
- Kilitlenme yönetimi için bir mekanizma sağlar.

- *Bir bilgisayarda CPU üzerinde yürütülen her program genel olarak bir "process" olarak adlandırılır.*

Giriş/Çıkış işlemleri (I/O operations)

- Çalışan bir process, bir dosya veya bir G/Ç aygıtı içerebilen G/Ç işlemi gerektirebilir.
- Verimlilik ve koruma için kullanıcılar genellikle G/Ç cihazlarını doğrudan kontrol edemez.
- Bu nedenle işletim sistemi çekirdeği, G/Ç cihazlarını kullanarak herhangi bir dosyanın okunması veya yazılması işlemlerine ve ayrıca gerektiğinde herhangi bir gerekli G/Ç cihazına erişime izin verir.
- Örneğin, bir dosyayı girdi olarak okuyan bir program, her farklı aygıt için programı değiştirmek zorunda kalmadan disket, sabit disk veya CD-ROM'daki bir dosyayı okuyabilmelidir.

Dosya sistemi yönetimi (File-system manipulation)

- Bir dosya sistemi kolay kullanım için dizinler halinde düzenlenir. Bu dizinler de dosyaları içerir.
- Dosya yönetimi ile ilgili olarak bir işletim sisteminin ana faaliyetleri şunlardır:

- kullanıcıya dosya oluşturması/silmesi için bir arayüz sağlar.
- kullanıcıya dizin oluşturması/silmesi ve dosyaları çoğaltması için bir arayüz sağlar.
- kullanıcıya bir dosyayı okuması/yazması için bir arayüz sağlar.
- dosyalar için izin yönetimi yapar, yani dosya sahipliğine bağlı olarak bu dosyalara veya dizinlere erişime izin verir veya vermez.
- İşletim Sistemi, dosya sisteminin yedeğini oluşturmak için bir arayüz sağlar.

İletişim (Communication)

- Bir process'in başka bir process ile bilgi alışverişi yapması gereken birçok durum vardır.
- Bu iletişim, aynı bilgisayarda yürütülen process'ler arasında veya bir bilgisayar ağı tarafından birbirine bağlı farklı bilgisayar sistemlerinde yürütülen process'ler arasında gerçekleşebilir.
- İletişim, *paylaşılan bellek yoluyla (shared memory)* veya veri paketlerinin işletim sistemi tarafından proces'ler arasında taşındığı *mesaj iletme (message passing) yoluyla* gerçekleştirilebilir.

Hata tespiti (Error detection)

- İşletim sistemi bilgisayar sisteminde meydana gelen her türlü hataları algılayabilmelidir.
- Bu hatalar işlemci, ana bellek, G/Ç aygıtı ya da kullanıcı programında meydana gelmiş olabilir.
- Meydana gelen hataya uygun gerekli eylem ya da düzeltme işlemi işletim sistemi tarafından gerçekleştirilir.
- *Örneğin bir kelime işlemci programından bir dosyanın açılarak yazıcıdan çıktısı alınması istenebilir.*
- *Fakat yazıcı bu işlemi gerçekleştirirken, yazıcıda kağıt bittiğine dair mesaj ya da yazıcının açık olmadığına dair gelen mesaj gibi.*

Kaynak tahsisi (Resource allocation)

- Bir çok kullanıcının ya da bir çok process'in aynı anda çalıştığı durumlarda elde bulunan tüm kaynakların paylaştırılması ve verimli kullanılması gerekir.
- Kaynak tahsisi, işlemci, bellek ve dosya depolama gibi kaynakların her process'e dinamik olarak tahsis edilmesi ve tahsisinin kaldırılmasıdır.
- Amaç, mevcut sınırlı kaynaklara bağlı olarak cevap süresini optimize edecek şekilde kaynakları tahsis etmektir. İşlemcinin process'lere tahsisi için işlemci zamanlama algoritmaları kullanılır.

Kaynak tahsisi (Resource allocation)

- *Örneğin, çoğu process verileri sabit disk gibi bir depolama aygıtına kaydetmesi gerekir.*

- *Aynı anda çalışan üç process olduğunu ve üç process'in de düzenli olarak veri kaydettiğini hayal edin.*

- *Process'ler, depolama aygıtı zaten kullanımdayken verileri kaydetmek için istekler gönderebilir. İşletim sistemi, üç process'in de veri kaydedebilmesi için bu istekleri yönetmelidir.*

Kullanıcı işlemleri (Accounting)

- Çok kullanıcılı ortamlarda hangi kullanıcının hangi haklara sahip olduğu ve sistem kaynaklarından ne kadar yararlanabileceği işletim sistemi tarafından yönetilir.

Koruma ve güvenlik (Protection and security)

- Koruma, sistem kaynaklarına tüm erişimin kontrol edilmesini içerir.
- Birkaç ayrı process eşzamanlı olarak yürütüldüğünde, bir process'in diğerlerine müdahale etmesi engellenmeli ve korunmalıdır.
- Sistemin güvenliği de önemlidir.
- Güvenlik, her kullanıcının sistem kaynaklarına erişim elde etmek için genellikle bir parola aracılığıyla kendini sisteme doğrulamasını istemekle başlar.
- Örneğin, çok kullanıcılı veya ağa bağlı bir bilgisayar sisteminde depolanan bilgilerin sahipleri bu bilgilerin kullanımını kontrol etmek isteyebilir.

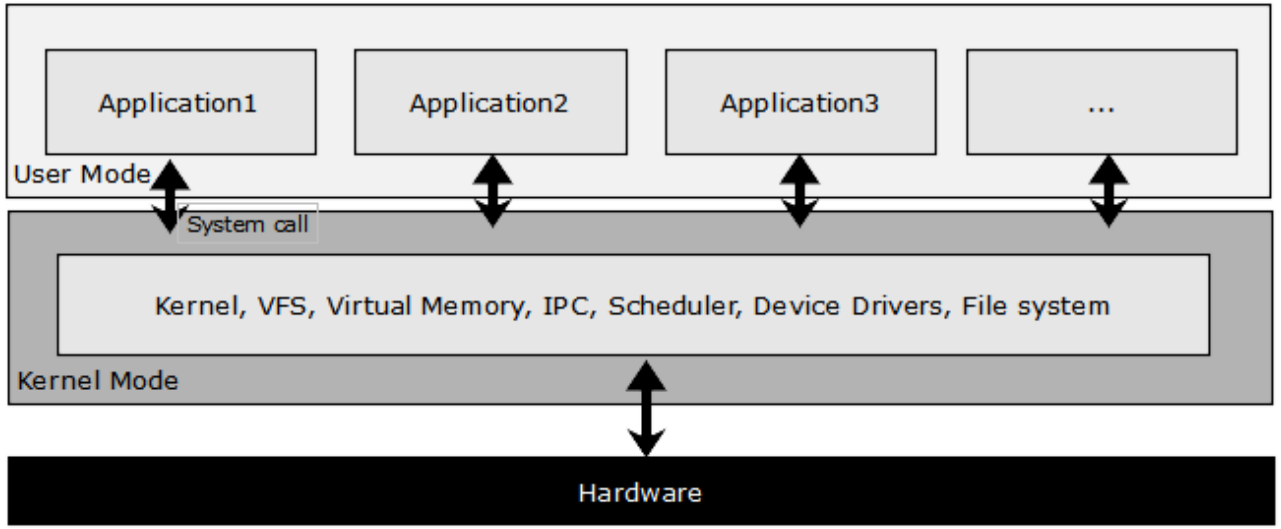
İşletim Sistemi Yapıları (Operating-System Structures)

- İşletim sisteminin yapısı temel olarak işletim sisteminin çeşitli ortak bileşenlerinin nasıl birbirine bağlandığına ve çekirdeğe nasıl dahil edildiğine bağlıdır.
- Buna bağlı olarak, işletim sistemleri aşağıdaki yapılarda tasarlanmışlardır:

- 1. Monolitik çekirdek mimarisi (Monolithic kernel architecture)
- 2. Mikro çekirdek Yapı (Micro-kernel structure)
- 3. Moduler Yapı (Modular structure / Loadable Kernel Modules)

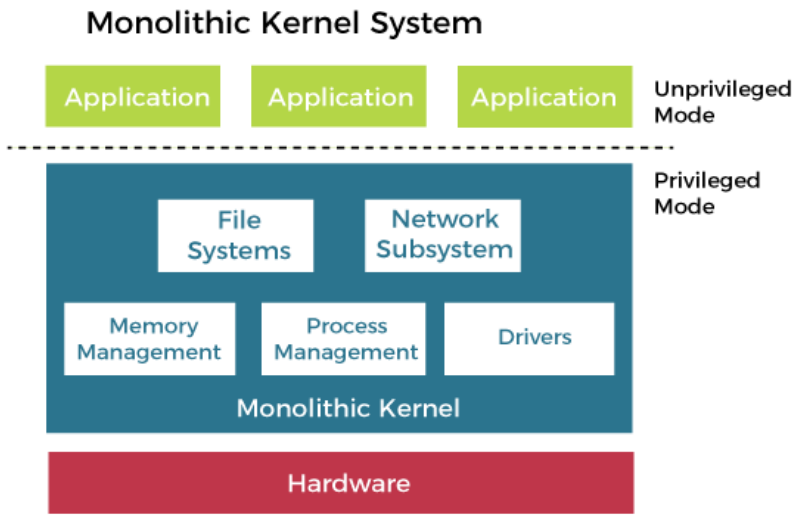
Monolitik çekirdek (Monolithic kernel)

- Monolitik çekirdek mimarisi, tüm temel işletim sistemi servislerinin (bellek yönetimi, process yönetimi, dosya sistemleri, sürücü yönetimi vb.) çekirdeğin bir parçası olarak çalıştığı bir mimariyi ifade eder.
- Monolitik yapıda, bütün bileşenler çekirdek içinde bulunur.
- **Kullanıcı ve çekirdek servisleri aynı bellek alanında uygulanır.**
- **Uygulamalar ile donanım arasındaki iletişim sistem çağrısı (system call) kullanılarak kurulur.**
- Monolitik mimariyi kullanan işletim sistemleri ilk kez 1970'lerde kullanıldı.



Monolitik çekirdek (Monolithic kernel)

- **Avantajları:**
- **Basitlik:** Monolitik çekirdeklerin tasarımı ve uygulanması diğer yapılara göre daha kolaydır.
- **Performans:** Bellek yönetimi, dosya yönetimi, işlemci zamanlama vb. servisler aynı adres alanı altında gerçekleştirildiği için oldukça hızlıdır.
- Sistem çağrıları (system calls) sırasında çekirdek ve kullanıcı modları arasında daha az bağlam değişimi (context switch) gerektirir, bu da performansı artırır.
- Çekirdeğin tüm bileşenleri birbirine doğrudan erişebilir, bu da esneklik ve hız açısından avantaj sağlar.

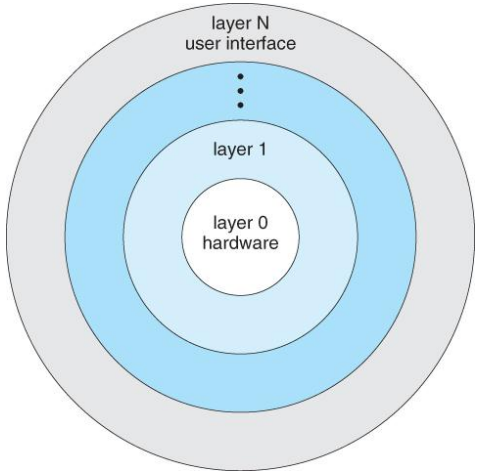


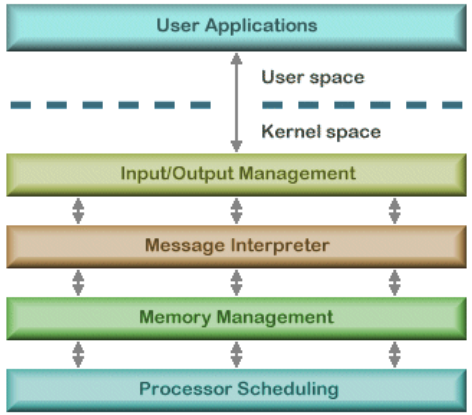
Monolitik çekirdek (Monolithic kernel)

- **Dezavantajları:**
- Çekirdek kodunda olası bir hata nedeniyle çekirdeğin herhangi bir bölümünün, çekirdeğin başka bir bölümüne ait belleğe erişmesi ve zarar görmesi mümkündür.
- Çekirdekte herhangi bir servis başarısız olursa, tüm sistemin çökmesine neden olur.
- Bu işletim sisteminde yazılmış kodun taşınması zordur.
- Monolitik yapıda yeni servisler eklemek ve kaldırmak çok zordur. Herhangi bir servis eklemek veya kaldırmak için tüm kodların yeniden yazılması ve yeniden derlenmesi gerekir.
- İşletim sistemi büyüdükçe monolitik çekirdeğin boyutu da büyür, bu da bakımı ve hata ayıklamayı zorlaştırır.

İşletim Sisteminin Katmanlı Yapısı (Layered Structure of Operating System)

- İşletim sisteminde modülerliği sağlamanın bir yolu katmanlı yaklaşımdır.
- Bu yaklaşımda en alt katman donanım, en üst katman ise kullanıcı arayüzüdür.
- Her bir üst katman, bir alt katmanın üzerine inşa edilmiştir.
- Dolaysıyla her üst katman yalnızca alt düzey katmanların işlevlerini kullanabilir.
- Bu yüzden, her katmanın dikkatlice tasarlanması gerekir.





İşletim Sisteminin Katmanlı Yapısı (Layered Structure of Operating System)

- **Avantajlar:**
- **Modülerlik:** Bu tasarım, her katmanın yalnızca gerçekleştirmesi planlanan görevleri yerine getirmesi nedeniyle modülerliği destekler.
- **Kolay hata ayıklama:** Katmanlar ayrı olduğundan hata ayıklamak çok kolaydır. Bir katmanda hata oluştuğunda, geliştirici, tüm hizmetlerin mevcut olduğu Monolitik sistemden farklı olarak, hata ayıklamak için yalnızca o belirli katmanı arayabilir.
- **Kolay güncelleme:** Belirli bir katmanda yapılan değişiklik diğer katmanları etkilemez. Güncellemesi daha kolaydır.
- **Donanıma doğrudan erişim yok:** Kullanıcı donanım hizmetlerini kullanabilir ancak doğrudan değiştiremez veya donanıma doğrudan erişemez.
- **Soyutlama:** Her katman kendi işlevleriyle ilgilidir. Dolayısıyla diğer katmanların işlevleri ve uygulamaları onun için soyuttur.

İşletim Sisteminin Katmanlı Yapısı (Layered Structure of Operating System)

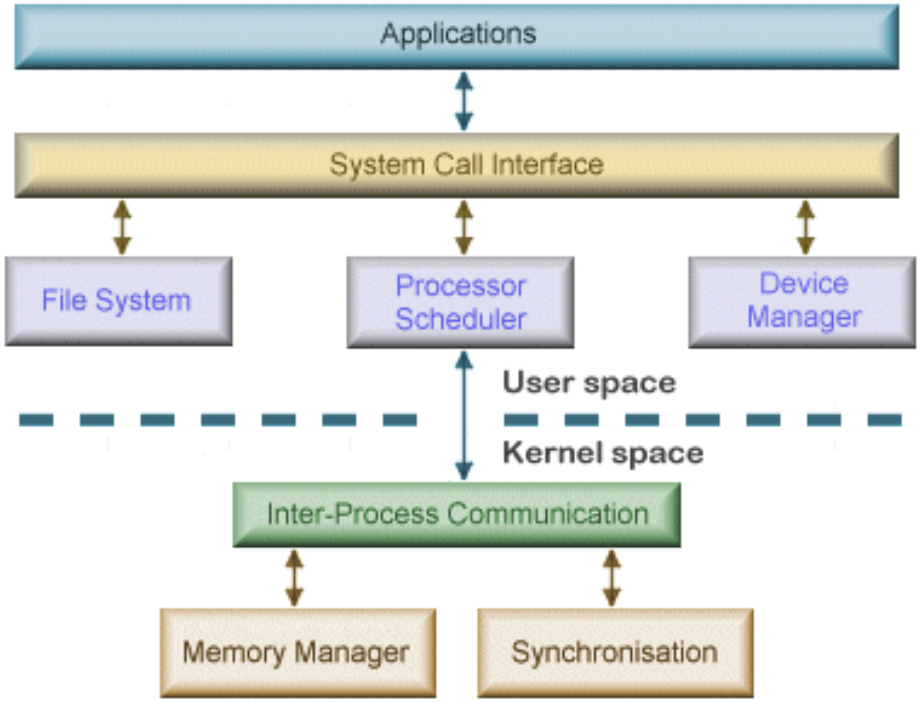
- **Dezavantajlar:**
- **Karmaşık katman tasarımı:** Bir katman yalnızca alt düzey katmanları kullanabildiğinden, katmanların uygun bir şekilde tanımlanması ve katmanların hangi sırayla yerleştirileceğine dikkatlice karar verilmelidir.
- **Yürütmede daha yavaş:** Bir kullanıcı process'inden gelen bir servis talebi, birçok servis katmanından geçmek zorunda kalabilir.
- Böylece daha hızlı olan Monolitik sistemin aksine tepki süresini artırır. Dolayısıyla katman sayısındaki artış çok verimsiz bir tasarıma yol açabilir.

Mikro çekirdek (Micro kernel)

- 1980'lerin ortalarında Carnegie Mellon Üniversitesi'ndeki araştırmacılar, mikro çekirdek yaklaşımını kullanarak çekirdeği modülerleştiren Mach adında bir işletim sistemi geliştirdiler.
- *Black, D., Golub, D. B., Julin, D. P., Rashid, R. F., Draves, R. P., Dean, R. W., Forin, A., Barrera, J., Tokuda, H., Malan, G. R., & Bohman, D. (1991). Microkernel operating system architecture and Mach. Journal of information processing, 14(4), 442-453.*
- Mikro çekirdek, işletim sisteminin çekirdeğinde yalnızca temel işlevselliğin sağlandığı bir işletim sistemi mimarisidir.
- Sonuç daha küçük bir çekirdektir.
- Mikro çekirdek mimarisi, karmaşıklığı azaltılmış hafif bir çekirdeğe sahiptir. Bu, işletim sistemini genişletmeyi kolaylaştırır çünkü tüm yeni servisler çekirdeği değiştirmeye gerek kalmadan kullanıcı alanına eklenir.

Mikro çekirdek (Micro kernel)

- Bu yapıda, yalnızca en önemli servisler çekirdek içinde uygulanır ve geri kalan işletim sistemi servisleri, kullanıcı adres alanında sistem uygulama programı aracılığıyla uygulanır.
- Bu nedenle mikro çekirdek, işletim sisteminin yalnızca en önemli hizmetlerinden sorumludur:
- Process'ler arası iletişim (Inter-Process Communication-IPC)
- Bellek yönetimi (Memory manager)
- Senkronizasyon (Synchronisation)
- **Kullanıcı modunda çalışan hizmetler, çekirdek ile iletişim kurmak için IPC mekanizmalarından olan mesaj geçişini (message passing) kullanır.**



Mikro çekirdek (Micro kernel)

- Mikro çekirdek (microkernel) mimarisine sahip bir işletim sisteminde de sistem çağrıları (system calls) kullanılır. Ancak, geleneksel monolitik çekirdek mimarilerinden farklı olarak, mikro çekirdeklerde sistem çağrılarının rolü ve işleyişi biraz farklıdır.
- Mikro çekirdeklerde sistem çağrıları, geleneksel monolitik çekirdeklere göre daha minimal ve optimize edilmiştir.
- Mikro çekirdekler, yalnızca temel işlevleri sağladığı için sistem çağrılarının sayısı azalır.
- İşletim sistemi servislerinin çoğu çekirdekte değil, kullanıcı modunda çalıştığı için, processler arası iletişim (Inter-Process Communication-IPC) mekanizmaları daha sık kullanılır.
- Sistem çağrıları doğrudan çekirdek servislerine erişmek için kullanılır, IPC ise farklı processlerin birbirleriyle veri paylaşmasını sağlamak için kullanılır.

Mikro çekirdek
(Micro kernel)

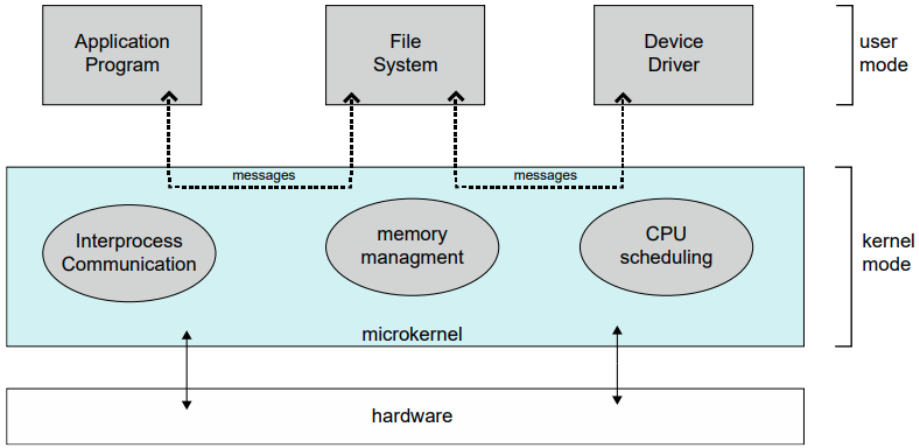
- Çekirdeği küçük ve ölçeklenebilir kılmak amacıyla çekirdek içinde yalnızca çok az sayıda servis modülü içerir. Böylece işletim sisteminin boyutu da azaltılır.
- **Kullanıcı ve çekirdek hizmetleri farklı bellek adres alanlarında uygulanır.** Birbirinden ayrı kullanıcı bellek adres alanı ve çekirdek bellek adres alanı vardır.
- Mikro çekirdek yapıda, çekirdeğin çeşitli servis modülleri kullanıcı adres alanına taşınmıştır, servis modülleri birbirinden etkili bir şekilde yalıtılmıştır, ancak servislerin birbirleriyle iletişim kurması için güvenli bir yol sağlanmıştır.

Mikro çekirdek
(Micro kernel)

- **Avantajlar:**
- Mikro çekirdek yapıyı yeni mimarilere taşımak, ölçeklendirmek ve genişletmek kolaydır.
- Herhangi bir yeni servis modülü eklenecekse, kullanıcı adres alanına eklenir ve dolayısıyla çekirdek alanında herhangi bir değişiklik yapılmasını gerektirmez.
- Çekirdeğin değiştirilmesi gerektiğinde, mikro çekirdek daha küçük bir çekirdek olduğundan değişiklikler daha az olma eğilimindedir.
- Mikro çekirdek aynı zamanda daha fazla güvenlik ve güvenilirlik sağlar, çünkü kullanıcı adres alanındaki servisler ve çekirdek servisleri izole edildiğinden herhangi bir kullanıcı adres alanı servisi başarısız olursa çekirdek servisleri bundan etkilenmez.

Mikro çekirdek (Micro kernel)

- **Dezavantaj:**
- Kullanıcı uygulaması ile kullanıcı adres alanında çalışan servis modülleri arasındaki iletişim "message passing" denilen yöntemle gerçekleştirilir.
- Uygulamalar ve işletim sistemi servisleri arasındaki iletişimin artan seviyesi sistem performansını oldukça düşürebilir.
- Kullanıcı modundan çekirdek moduna geçişler daha sık olduğu için daha fazla bağlam değişimi (context switch) gerektirir buda ekstra yük getirir.
- Mikro çekirdeklerdeki çeşitli servis modüllerini yönetmenin karmaşıklığı, geliştiriciler için zorluk oluşturabilir.
- Etkileşimleri koordine etmek ve her servis modülünün doğru çalışmasını sağlamak, dikkatli bir tasarım ve kapsamlı testler gerektirir.



Mikro Çekirdek ve Monolitik Çekirdek Arasındaki Temel Farklar

- Mikro çekirdeğin kullanıcı hizmetleri ile çekirdek hizmetleri farklı adres alanlarında uygulanır , monolitik çekirdeğin ise hem kullanıcı hizmetleri hem de çekirdek hizmetleri aynı adres alanında uygulanır.
- Mikro çekirdeğin boyutu küçüktür çünkü çekirdek adres alanında yalnızca çekirdek hizmetleri bulunur. Ancak, monolitik çekirdeğin boyutu, hem çekirdek hizmetleri hem de kullanıcı hizmetleri aynı adres alanında bulunduğu için mikro çekirdekten nispeten daha büyüktür .

Mikro Çekirdek ve Monolitik Çekirdek Arasındaki Temel Farklar

- Monolitik çekirdeğin yürütülmesi daha hızlıdır çünkü uygulama ile donanım arasındaki iletişim **sistem çağrısı (system call)** kullanılarak kurulur . Öte yandan, mikro çekirdeğin yürütülmesi yavaştır çünkü uygulama ile sistemin donanımı arasındaki iletişim **mesaj geçişi (message passing)** yoluyla kurulur.
- Mikro çekirdeği genişletmek kolaydır çünkü yeni servis çekirdek alanından izole edilmiş kullanıcı adres alanına eklenecektir, bu nedenle çekirdeğin değiştirilmesi gerekmez. Monolitik çekirdekte ise durum tam tersidir, monolitik çekirdeğe yeni bir servis eklenecekse tüm çekirdeğin değiştirilmesi gerekir.

Mikro Çekirdek ve Monolitik Çekirdek Arasındaki Temel Farklar

- Mikro çekirdek, monolitik çekirdekten daha güvenlidir çünkü mikro çekirdekte bir servis başarısız olursa işletim sistemi etkilenmez. Öte yandan, monolitik çekirdekte bir servis başarısız olursa tüm sistem başarısız olur.
- Monolitik çekirdek tasarımı daha az kod gerektirir, bu da daha az hataya yol açar. Öte yandan, mikro çekirdek tasarımı daha fazla kod gerektirir, bu da daha fazla hataya yol açar.

Moduler çekirdek Yapı
(Modular kernel structure / Loadable kernel modules)

- Modular kernel, monolithic kernel ile microkernel'in avantajlarını birleştiren esnek bir çekirdek mimarisidir. Bu tür tasarım modern işletim sistemlerinde kullanılır.
- **Çalışma zamanı (runtime) sırasında, temel çekirdek olarak da bilinen çekirdeği genişletmek için dinamik olarak yüklenebilir çekirdek modülleri bulunur.**
- Tüm çekirdek modüllerini belleğe yüklemek yerine, çekirdek modülleri isteğe bağlı olarak çalışma zamanında belleğe dinamik olarak yüklenebilir veya kaldırılabilir.
- Çekirdeğe yeni her modül eklendiğinde çekirdeğinin tamamını yeniden derlemeye gerek kalmaz.

Moduler çekirdek Yapı
(Modular kernel structure / Loadable kernel modules)

- **Avantajlar :**
- **Performans:** Çekirdek belleğinde yalnızca gerekli olan servis modüllerini tutmak, çekirdeğin bellek alanını azaltır ve genel performansını artırır.
- **Hızlı önyükleme:** Genellikle önyükleme süresini azaltır, önyükleme sırasında her şeyi yüklemek zorunda değildir, daha hızlı geliştirme süresi sağlar.
- **Kullanışlı:** Yeni eklenen modüller için yeniden başlatmaya ve çekirdeği derlemeye gerek yoktur.
- **Verimli:** Mikro çekirdekten farklı olarak servis modülleri arasında iletişim için "message passing" olayına gerek yok.
- **Esnek:** Her bir çekirdek alt katmanının tanımlanmış, korumalı arayüzlere sahip olması bakımından katmanlı bir sisteme benzer; ancak herhangi bir modülün diğer herhangi bir modülü çağırabilmesi açısından katmanlı bir sistemden daha esnektir.

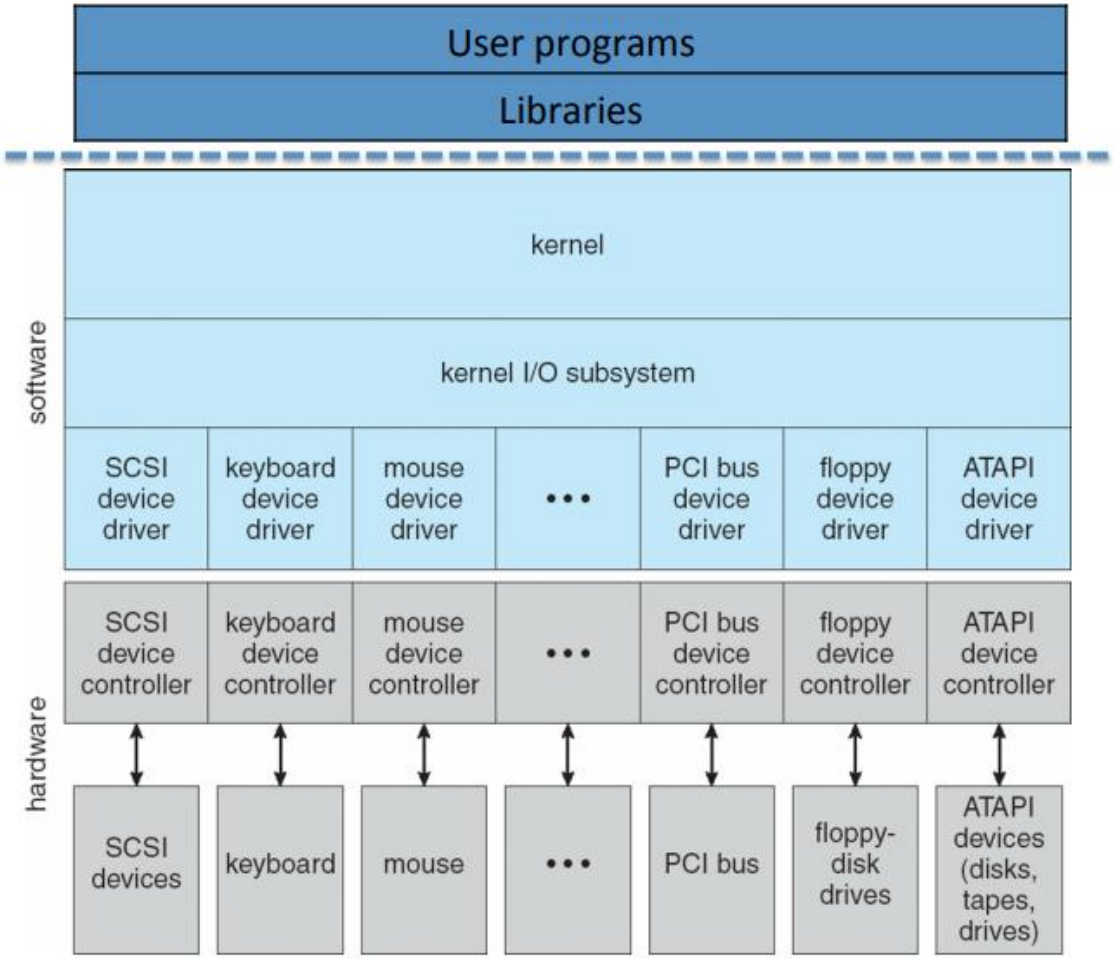
Moduler çekirdek Yapı
(Modular kernel structure / Loadable kernel modules)

- **Dezavantajı**
- Parçalanma cezası (fragmentation penalty), monolitik bir çekirdeğe göre modüler bir çekirdeği tercih etmenin büyük bir dezavantajıdır.
- Parçalanma cezası, çalışan çekirdeğe her modül kodu eklendiğinde ve çıkarıldığında, çekirdeğin bellek uzayındaki parçalanması anlamına gelir.
- Bu durum, daha fazla TLB (Transaction Lookaside Buffer) nedeniyle performans düşmesine yol açar. TLB sanal bellek adreslerinin fiziksel bellek adreslerine dönüştürülmesini hızlandıran bir donanım bileşenidir.

Çekirdek ve Kullanıcı Modu (Kernel and User Mode)

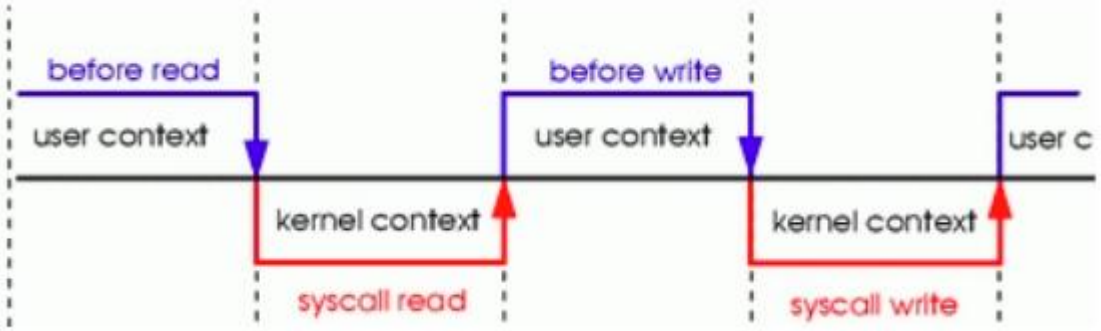
- **Bir işletim sistemi, kullanıcı modu (user mode) ve çekirdek modu (kernel mode) olmak üzere iki modda çalışır.**

- Kullanıcı modu ile çekirdek modu arasındaki temel fark, kullanıcı modunun uygulamaların çalıştığı mod ve çekirdek modunun işletim sisteminin donanım kaynaklarına erişirken girdiği ayrıcalıklı mod olmasıdır.



Çekirdek ve Kullanıcı Modu (Kernel and User Mode)

- İşletim sisteminin en kritik görevleri çekirdek modunda yürütülür.
- İşletim sistemi bir uygulama yazılımını çalıştırırken, kullanıcı modundadır. Uygulama yazılımı donanım kullanma talebinde bulunduktan sonra işletim sistemi çekirdek moduna girer.
- İşletim sistemi sık sık kullanıcı modu ve çekirdek modu arasında geçiş yapar.



Çekirdek (Kernel) Mode

- İşletim sistemi çekirdeği, donanım ve uygulama yazılımları için bir ara katman yazılımı olarak çalışır.
- Çekirdek bilgisayar sistemi açılırken ana belleğe yüklenir ve sistem kapatılıncaya kadar ana bellekte kalır.
- Çekirdek gereksinim duyulan diğer işletim sistemi parçalarını ikincil bellekten ana belleğe yükler.
- Çekirdek modu, sistem modu (system mode) veya ayrıcalıklı mod (privileged mode) olarak da adlandırılır ve işletim sisteminin düşük seviyeli güvenilir işlevleri için kullanılır.

Çekirdek (Kernel) Mode

- Bir process kullanıcı modunda yürütülürken ve bu process bellek , yazıcı vb. gibi donanım kaynaklarının kullanılmasını gerektiriyorsa, bu process işletim sisteminin çekirdeğine bir istek göndermelidir.
- Bu istekler **sistem çağrıları (system call)** aracılığıyla yapılır. Ardından işletim sistemi kullanıcı modundan çekirdek moduna geçiş yapar.
- Görev tamamlandığında, çekirdek modundan kullanıcı moduna döner.
- Tüm process'leri çekirdek modunda çalıştırmak mümkün değildir çünkü bir process başarısız olursa tüm işletim sistemi başarısız olabilir.

Çekirdek (Kernel) Mode

- Çekirdek modda çalışan komutlara imtiyazlı-ayrıcalıklı komutlar (privileged instructions) denilmektedir.

- Çekirdek mod seviyesinde çalışan kod, donanıma ve sistemin tamamına tam yetki ile müdahale edebilir.

- Eğer bir kod çekirdek modunda çalıştırılıyorsa işlemci çalışmakta olan koda yönelik hiçbir erişim kontrolü uygulamaz.

Kullanıcı (User) Mode

- Uygulama programları ise, kullanıcı modu (user mode) denilen daha kısıtlı bir işlem seviyesinde çalışır.

- Bu mod, daha az ayrıcalıklı mod (less privileged mode), bağımlı mod (slave mode) veya kısıtlı mod (restricted mode) olarak adlandırılır.

- Kullanıcı modunda çalışan process'ler bellekte ancak kendileri için ayrılan alanlara erişebilirler ve çekirdeğe ait adres alanına erişemezler.

Kullanıcı (User) Mode

- Güvenli bir sistemde sıradan process'lerin bazı makina komutlarını kullanamaması gerekir.
- Kullanıcı mod, bazı kısıtlanmış işlemci komutlarını çalıştıramaz ve donanım kaynaklarına doğrudan müdahale edemez.
- Kullanıcı modda sadece sistem yapılandırmasına zarar vermeyecek makina komutları işletilebilir.
- Kullanıcı modenda, her uygulama ayrı ayrı çalışır ve bir uygulama çökerse, çökme o uygulamayla sınırlıdır. Diğer uygulamalar ve işletim sistemi çöken bir uygulamadan etkilenmez.

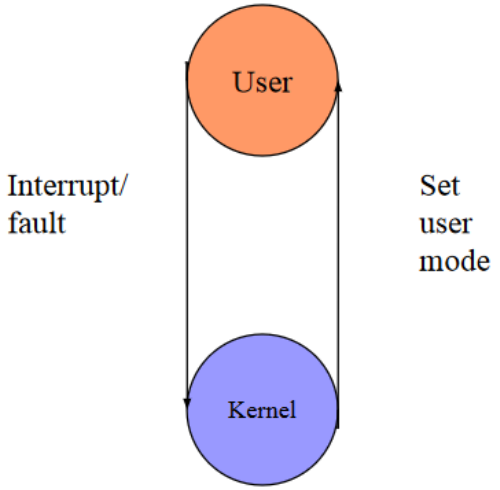
Donanımsal Koruma

- İşlemci çekirdek modda iken donanımsal her tür komutu çalıştırır.

- Kullanıcı modda **her komut** doğrudan yürütülecek olsaydı;
- Sistem yapılandırması değişebilir,
- Yetkisiz belleğe erişilebilir,
- Yetkisiz Giriş/Çıkış yapılabilir,
- vb.

Donanımsal Koruma

- **Koruma mekanizmasına sahip olan işlemcilerde çekirdek modu (kernel mode) ve kullanıcı modu (user mode) olmak üzere en azından iki çalışma modu bulunur.**
- İşlemcide, çalışma modunu gösteren bir **mod biti** bulunmaktadır.
- İşlemci, ne tür kodun çalıştığına bağlı olarak iki mod arasında geçiş yapar.
- Uygulamalar kullanıcı modunda ve ana işletim sistemi bileşenleri çekirdek modunda çalışır.



Geçerli modu belirtmek için işlemci donanımına eklenen mod biti: kernel (0) veya user (1).

Bir kesme veya trap oluştuğunda, donanım çekirdek moduna geçer.

Donanımsal Koruma
İşlemci modları ve işletim sistemi

- **Çift modlu çalışma :**
- İşlemci mimarisinde, en az iki işletim modu arasında ayrım yapmak için donanım desteği sağlanır:
- **1. çekirdek modu (kernel mode)** - yürütme işlemi, işletim sistemi çekirdeği adına yapılır.
- **2. kullanıcı modu (user mode)** - yürütme bir kullanıcı process'i adına yapılır.

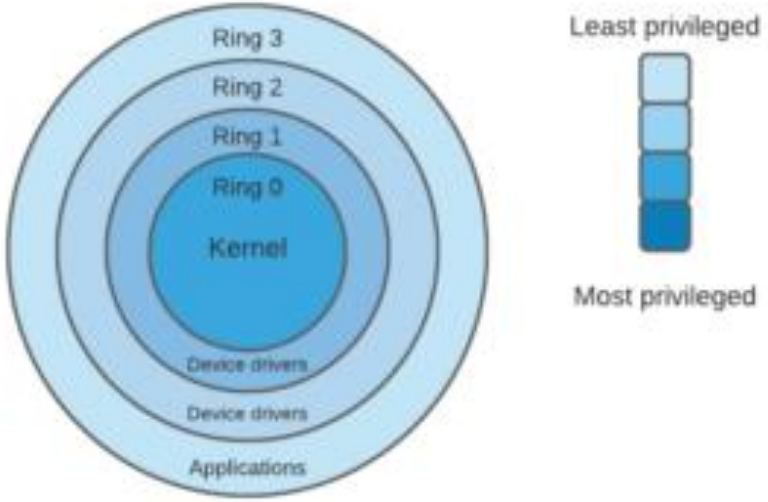
- Her zaman en az bir kullanıcı modu tanımlanır, ancak bazı işlemci mimarileri , genellikle bir ayrıcalık hiyerarşisi ile birden çok kullanıcı modunu destekleyebilir.

Donanımsal Koruma
İşlemci modları ve işletim sistemi

- Genel olarak; işlemcinin **mod biti** işletim sisteminin koruma haklarından biridir.
- İşletim sistemi çekirdek modda çalıştığında, kullanıcı moduna göre belleğe ve öncelikli komut kümesine ulaşmakta daha fazla haklara sahip olmaktadır.
- Kullanıcı modunda çalışan bir process işletim sistemi servislerini çağırdığında işlemci hemen çekirdek moda, mod bitini kullanarak geçer ki bu duruma sistem çağrısı (system call) denilmektedir.

Intel işlemci modları

- Tipik bir modern işlemci, farklı ayrıcalık seviyelerini temsil eden farklı koruma halkaları sağlar.
- Örneğin Intel x86 işlemcilerinde 0, 1, 2, 3 biçiminde belirtilen 4 koruma halkası bulunur.
- Daha düşük numaraya sahip bir halkanın daha yüksek ayrıcalıkları vardır. 0 en yüksek önceliği, 3 en düşük önceliği belirtir.



Intel işlemci modları

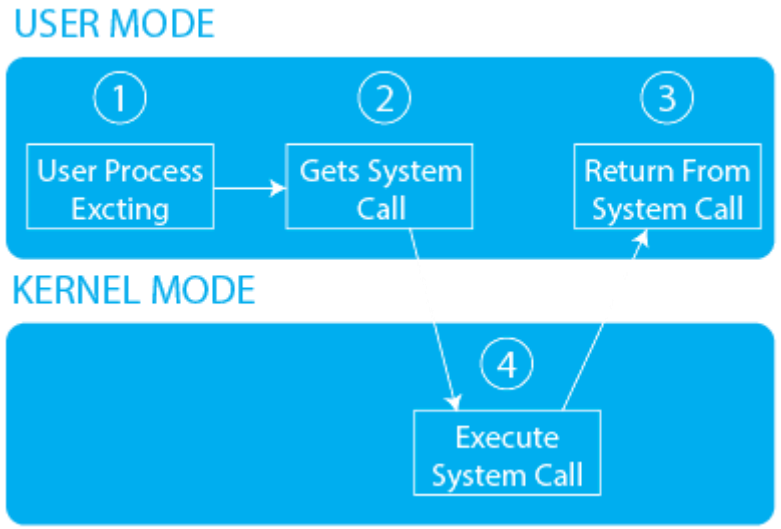
- Intel X86 işlemciler için **Linux/Unix/Windows işletim sistemlerinde**, 0 çekirdek modunu ve 3 kullanıcı modunu temsil eder.

- Intel 8086/8088 gibi eski işlemcilerde mod biti bulunmadığı için çekirdek ve kullanıcı modu arasında bir ayrım yapılmamaktadır.

- DOS (Disk operating system) ve diğer benzer basit işletim sistemleri ve gömülü aygıtlar sürekli olarak çekirdek modunda çalışır.

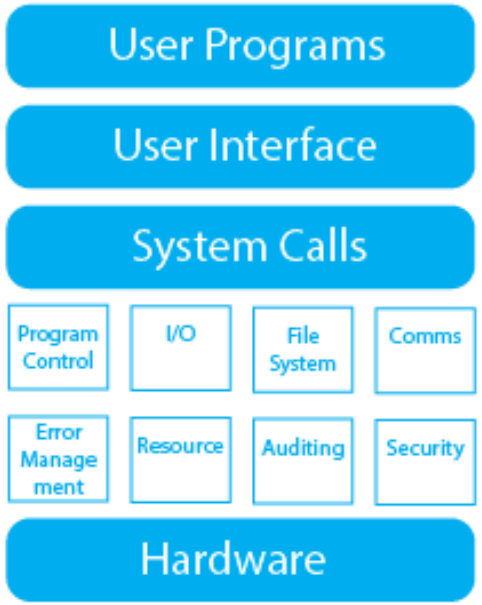
Sistem Çağrıları (System Calls)

- Sistem çağrısı, bir kullanıcı process'inin, üzerinde çalıştığı işletim sisteminin çekirdeğinden bir hizmet isteme yöntemidir.
- Hizmet genellikle, G/Ç işlemleri gibi sadece çekirdek ayrıcalığında olan donanımsal işlemlerdir.
- Kullanıcı düzeyindeki programlar çekirdek sistemine yalnızca sistem çağrıları kullanarak erişebilir.
- Bir program sistem çağrısı yaptığında kontrolü kullanıcı modundan çekirdek moduna aktarıyor demektir.



Sistem Çağrıları (System Calls)

- Sistem çağrıları, işletim sistemleri tarafından sunulan bir arayüzdür. Bu arayüz, kullanıcı mod ile çekirdek mod arasında yer alır.
- Çekirdek modu, işletim sisteminin yürütüldüğü ayrıcalıklı çalışma modudur ve tüm donanım kaynaklarına erişime sahiptir.
- Donanım üzerinde gerçekleştirilecek işlemler sistem çağrıları kullanarak gerçekleştirir. Bu sayede olası sistem hatalarından kaçınılmış olur.



İşletim Sisteminde neden Sistem Çağrılarına ihtiyacımız var?

- İşletim sistemindeki sistem çağrıları, uygulamaların temel donanımla etkileşime girmesi için standartlaştırılmış bir araç sağladıklarından sistemin güvenliğini, kararlılığını ve performansını sağlar.

- Ayrıca uygulamaların kaynaklara erişmesini ve process'leri işletim sistemine devretmesini mümkün kılar ve uygulamaların farklı platformlar arasında taşınabilirliğini sağlamaya yardımcı olurlar, bu nedenle işletim sistemi içinde sistem çağrılarına ihtiyaç duyulur.

Sistem Çağrıları (System Calls)

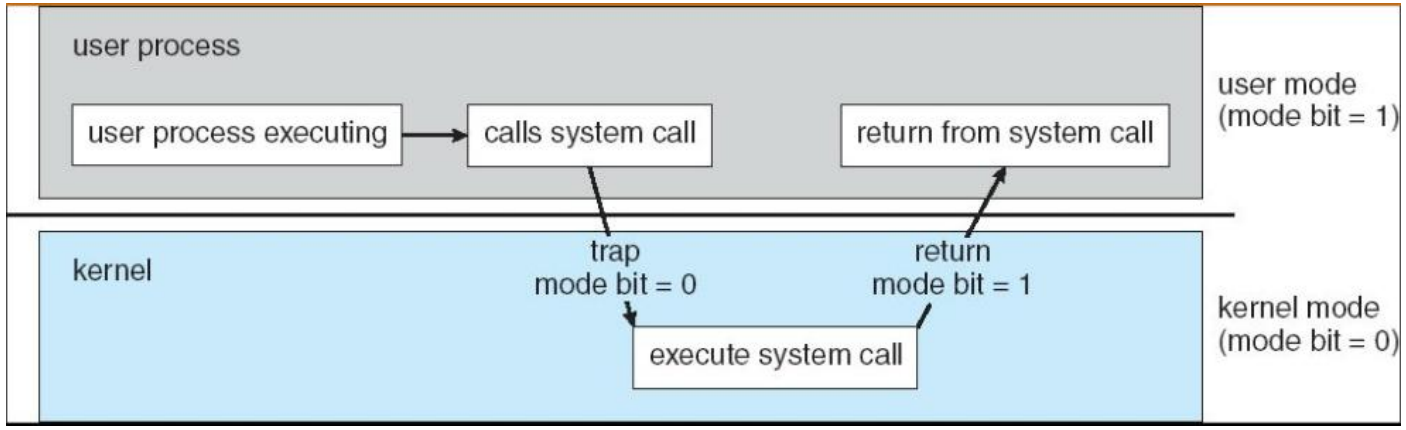
- Genel olarak, sistem çağrıları aşağıdaki durumlarda gereklidir:

- Dosyaların oluşturulması veya silinmesi, dosyalardan okuma ve yazma
- Yeni process'lerin oluşturulması ve yönetimi.
- Ağ bağlantıları ayrıca sistem çağrıları gerektirir. Buna paket gönderme ve alma dahildir.
- Yazıcı, tarayıcı vb. donanım aygıtlarına erişim, bir sistem çağrısı gerektirir.

- Kullanıcı modunda uygulamaların içerdiği ayrıcalıklı komutların çalıştırılması, **sistem çağrıları** ile yerine getirilir.

Sistem Çağrıları (System Calls)

- Bir kullanıcı uygulaması bir sistem çağrısı oluşturduğunda, önce çekirdekten izin almalıdır.
- Bunu, **"trap"** adı verilen, mevcut process'i duraklatan ve kontrolü çekirdeğe aktaran bir kesme isteği (interrupt request) kullanarak gerçekleştirir.
- **Trap**, bir hata veya bir kullanıcı isteği sonucunda oluşturulan kesme (interrupt) işlemidir.
- Bir kullanıcı uygulaması, bir **"trap"**'a neden olan bir sistem çağrısı yapabilir.



Sistem Çağrıları (System Calls)

- Sistem çağrıları doğrudan kullanılabileceği gibi, genellikle yüksek seviyeli API’ler aracılığıyla dolaylı olarak kullanılır.
- Doğrudan sistem çağrısı, genellikle assembler'da düşük seviyeli programlama gerektirir.
- Kullanıcının hedef mimariyi bilmesi gerekir, CPU'dan bağımsız kod oluşturamaz.
- Uygulama geliştiricilerin genellikle sistem çağrılarına doğrudan erişimi zor olduğu gibi, tehlikeli de olabilir.
- Bunun yerine Sistem çağrıları, işletim sisteminin API'sinde tanımlanır. Programcılar, doğrudan sistem çağrıları yerine API kullanmayı tercih ederler.

- *Uygulama Programlama Arayüzü (Application Program Interface -API)*

Sistem Çağrıları (System Calls)

- **Sistem çağrıları için neden API tercih edilmeli?**
- API'ler programın her sistem üzerinde çalışabilmesini sağlar. Böylece daha fazla program taşınabilirliği sağlanır.
- İşletim sistemi çekirdeği ile etkileşime girilmediği için çekirdeğe bir tür Güvenlik/Koruma sağlar.
- Programcıya birden fazla sistem işlemini kısa fonksiyonlarla yapabilme olanağı sağlar. Programcı API fonksiyonunun içeriğiyle ilgilenmez. Fonksiyonun parametrelerini ve dönüş değerini bilmesi yeterlidir.
- Gerçek sistem çağrısına kıyasla API kullanım kolaylığı sağlar. API'ler programlamayı oldukça kolaylaştırır.
- Ancak bazı özel işlemler için doğrudan sistem çağrılarını kullanmak da gerekebilir.

Sistem Çağrıları (System Calls)

- Bir restoranın müşterisi olduğunuzu hayal edin. Biraz yemek sipariş etmek istiyorsun.
- **Doğrudan Sistem Çağrısı :** Bu, doğrudan şeften (işletim sistemi çekirdeğini temsil eden) size özel bir şey pişirmesini istemeniz gibidir. Siz şefe ne istediğinizi söylüyorsunuz, şef de sizin için hazırlıyor. Burada isteğinizi yerine getirebilecek kişiyle doğrudan etkileşime giriyorsunuz.
- **API ile Sistem Çağrısı :** Şimdi restoranın (API'yi temsil eden) bir menüsü olduğunu varsayalım. Menü, mevcut tüm yemekleri listeler ve bunların nasıl sipariş edileceğine dair bir dizi talimat sağlar. Doğrudan şefle konuşmak yerine menüye bakarsınız, ne istediğinize karar verirsiniz ve ardından garsona (API'yi temsil eden) seçiminizi söylersiniz. Garson daha sonra siparişinizi şefe iletir ve yemeğinizi size getirir.
- Her iki yaklaşım da size istediğiniz yemeği verir, ancak farklı düzeylerde etkileşim ve soyutlama içerirler.

Sistem Çağrıları (System Calls)

- **Yaygın API standartları:**
- Windows, **Win32 API** kullanır.
- Unix, Linux tabanlı sistemler ise **POSIX API** kullanırlar.
- Java Sanal Makinası (JVM) ise **Java API** Kullanır.
- Unix, Linux tabanlı sistemlerde genellikle sistem çağrısı ile çağrılacak olan kütüphane fonksiyonunun ismi aynıdır.
- Bu isimler POSIX API tarafından tanımlanmıştır. Yani sistem çağrıları ile kütüphane prosedürleri arasında bire- bir ilişki vardır.
- Windows'ta durum tamamen farklıdır. Kütüphane çağrıları ve gerçek sistem çağrıları oldukça ayrıktır. Microsoft Win32 API adını verdiği bir prosedür kümesi tanımlamıştır.

Sistem Çağrıları (System Calls)

- POSIX, UNIX implementasyonları arasında yazılım birliğini sağlayabilmek amacı ile 1988 yılında IEEE tarafından ortaya çıkarılan bir standarttır.
- POSIX, yeniden kodlamaya gerek kalmadan farklı bilgisayar sistemlerine taşınabilecek uygulamalara ve programlara sahip olma gereksiniminden gelişti.
- POSIX API'sini bulunduran işletim sistemlerine "POSIX uyumlu işletim sistemi" denmektedir.
- POSIX API'si; aslen UNIX sistem çağrılarına ve C standart kütüphanesine dayanır.

Sistem Çağrıları (System Calls)

- Microsoft, Win32 API adı verilen ve programcıların işletim sistemi hizmetlerini almak için kullanması beklenen bir dizi prosedür tanımlamıştır.
- Bu arayüz Windows 95'ten beri Windows'un tüm sürümlerinde (kısmen) desteklenmektedir.
- Microsoft, API arayüzünü gerçek sistem çağrılarından ayırarak mevcut programları geçersiz kılmadan gerçek sistem çağrılarını zaman içinde (sürümden sürüme bile) değiştirme yeteneğini korur.
- Windows’un sistem çağrıları ve işletim sistemi kullanıcı değişikliklerine kapalıdır ve kullanıcılardan gizlenir.

Sistem Çağrıları (System Calls)

- Win32 API çağrılarının sayısı son derece fazladır ve binlerle ifade edilmektedir.
- Ayrıca, bunların birçoğu sistem çağrılarını çağırırken, önemli bir kısmı tamamen kullanıcı alanında gerçekleştirilir.
- Sonuç olarak, Windows ile neyin sistem çağrısı (yani çekirdek tarafından gerçekleştirilen) ve neyin sadece kullanıcı uzayı kütüphane çağrısı olduğunu görmek imkansızdır.
- Aslında, Windows'un bir sürümünde sistem çağrısı olan bir şey, farklı bir sürümde kullanıcı modunda yapılabilir ve bunun tersi de geçerlidir.

Sistem Çağrısı Çeşitleri

Sistem Çağrısı Tipleri

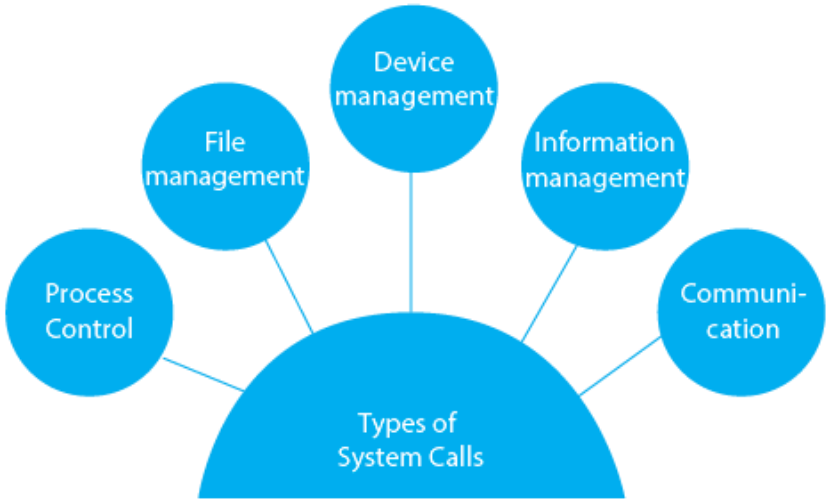
Process Control : process oluşturma, process sonlandırma vb.

File Management : dosya oluşturma, dosya okuma, dosyaya yazma vb.

Device Management : aygıt tamponlarından okuma, aygıt tamponlarına yazma vb. gibi aygıt manipülasyonlarından sorumludur.

Information Management : işletim sistemi ve kullanıcı programı arasındaki bilgi aktarımı.

Communication : process'ler arası iletişim için kullanışlıdır. Ayrıca bir iletişim bağlantısı oluşturmak ve silmekle de ilgilenirler.



Sistem Çağrısı Çeşitleri

Types of System Calls	Windows (Win32)	Linux (POSIX)
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()

Sistem Çağrıları (System Calls)

Bir dosyayı açmak için kullanılan C dilindeki fopen() fonksiyonu, altta yatan open() sistem çağrısını kullanır.

```
#include <stdio.h>
int main() {
FILE *file = fopen("test.txt", "r");
if (file == NULL) {
perror("Dosya açılamadı");
return 1;
}
fclose(file);
return 0;
}
```

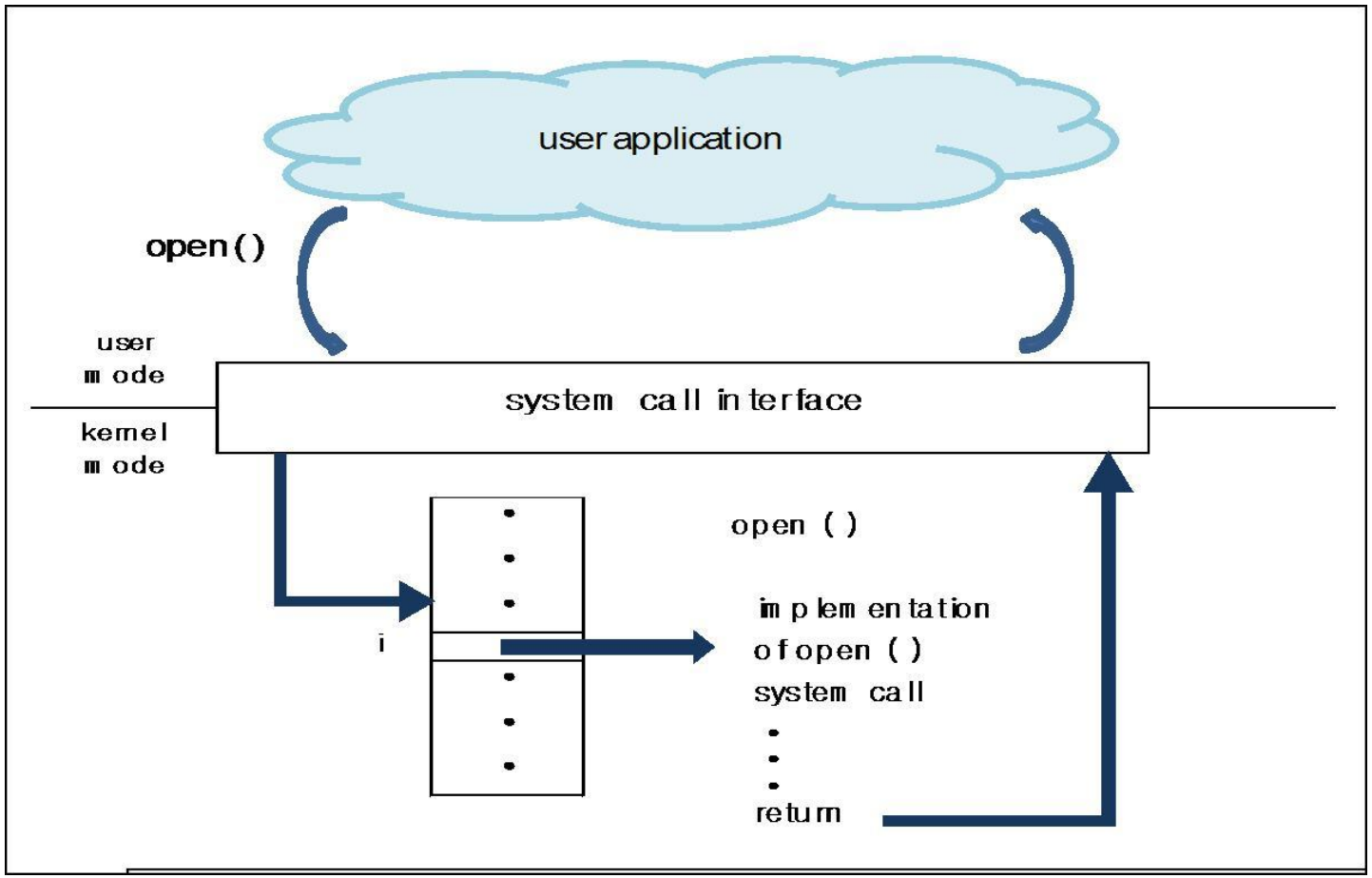
- Bu kod çalışırken arka planda şu işlemler gerçekleşir:
- fopen() API fonksiyonu, standart C kütüphanesi (glibc) içinde tanımlıdır.
- fopen(), çekirdek seviyesindeki open() sistem çağrısını kullanır.
- open() sistem çağrısı çekirdek moduna geçerek işletim sisteminin dosya sistemine erişim sağlar.
- İşletim sistemi, belirtilen dosyayı açıp bir dosya tanıtıcısı (file descriptor) döndürür.
- fopen(), dönen değeri işleyerek kullanıcıya bir FILE* nesnesi sunar.

Sistem Çağrıları (System Calls)

Sistem çağrıları sayesinde yazılımcı doğrudan donanıma müdahale etmez.

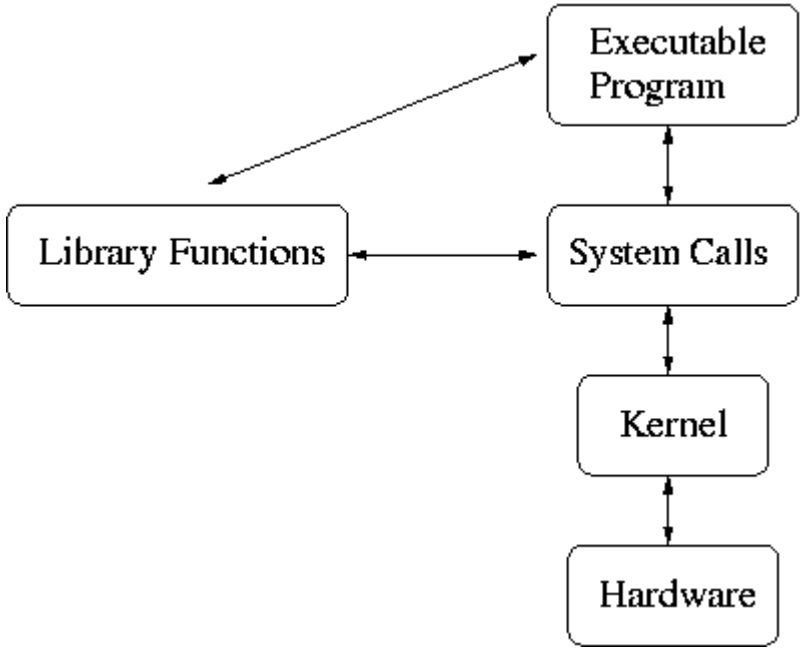
Donanım üzerinde gerçekleştireceği işlemi sistem çağrısı kullanarak gerçekleştirir.

Yandaki şekil, Linux işletim sisteminin **open ()** sistem çağrısını yapan bir kullanıcı uygulamasının nasıl ele aldığını göstermektedir.



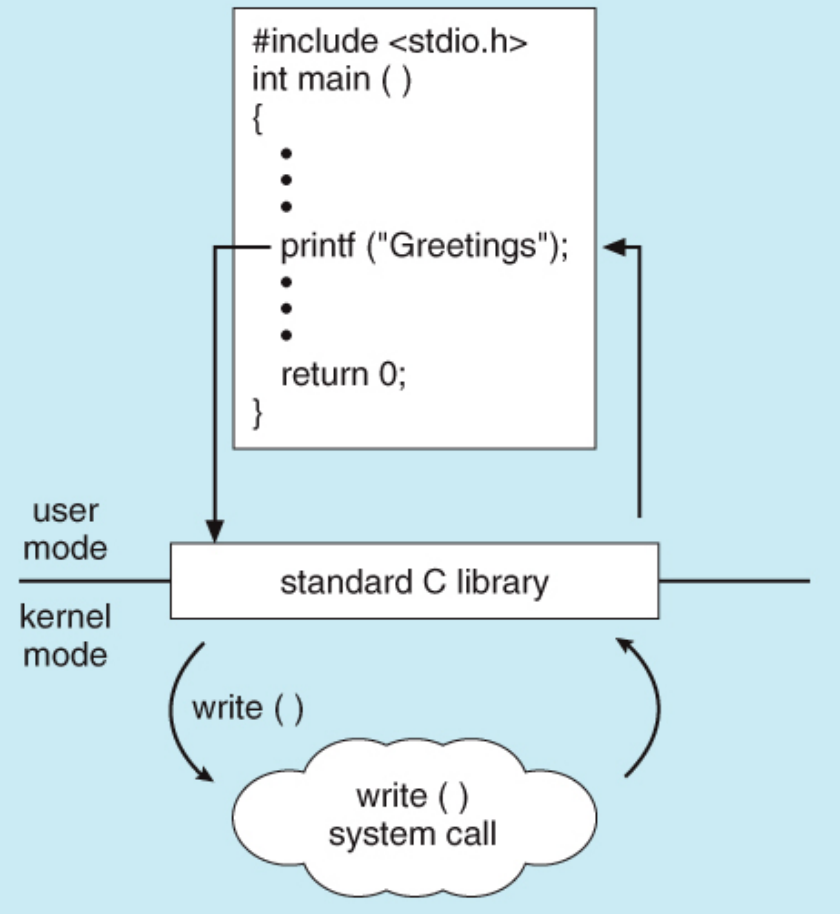
Sistem Çağrıları (System Calls)

- Yüksek seviyeli diller sistem çağrılarının kullanımını kolaylaştırır ve sistem çağrısı davranışını tanımlar.
- Sistem çağrısı, API aracılığıyla kullanıcı programlarına işletim sisteminin servislerini sağlar.
- Uygulama geliştirici, işletim sistemindeki API tarafından sağlanan fonksiyonları (dosya aç, oku, yaz, ...vs.) kullanır.
- Programcı, bu fonksiyonlara programlama dilinin sağladığı kütüphaneler aracılığıyla erişir.
- Dolayısıyla, yüksek seviyeli diller sistem çağrılarının ayrıntılarını soyutlar, ancak sonuçta işletim sistemiyle daha düşük seviyede etkileşim kurmak için temeldeki POSIX veya Win32 API'sine güvenirler.



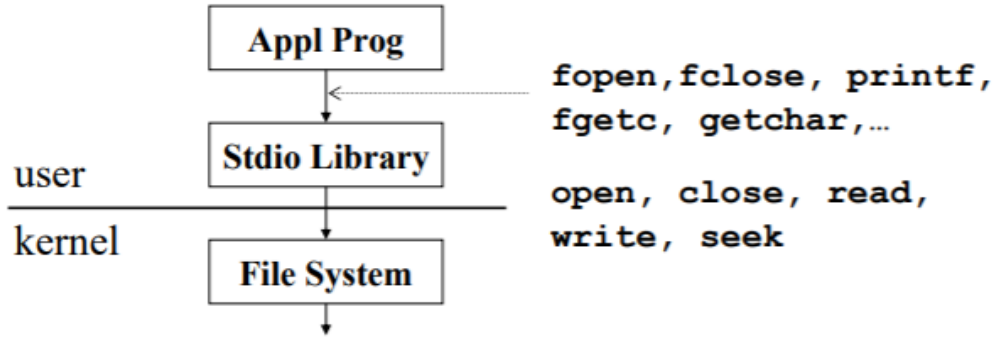
Sistem Çağrıları (System Calls)

- Bir sistem çağrısı yapmanın gerçek mekaniği makineye son derece bağlı olduğundan ve genellikle assembly kodunda ifade edilmesi gerektiğinden, C gibi bir programlama dilinden ve diğer dillerden de sistem çağrıları yapmayı mümkün kılmak için bir prosedür kütüphanesi sağlanmıştır.

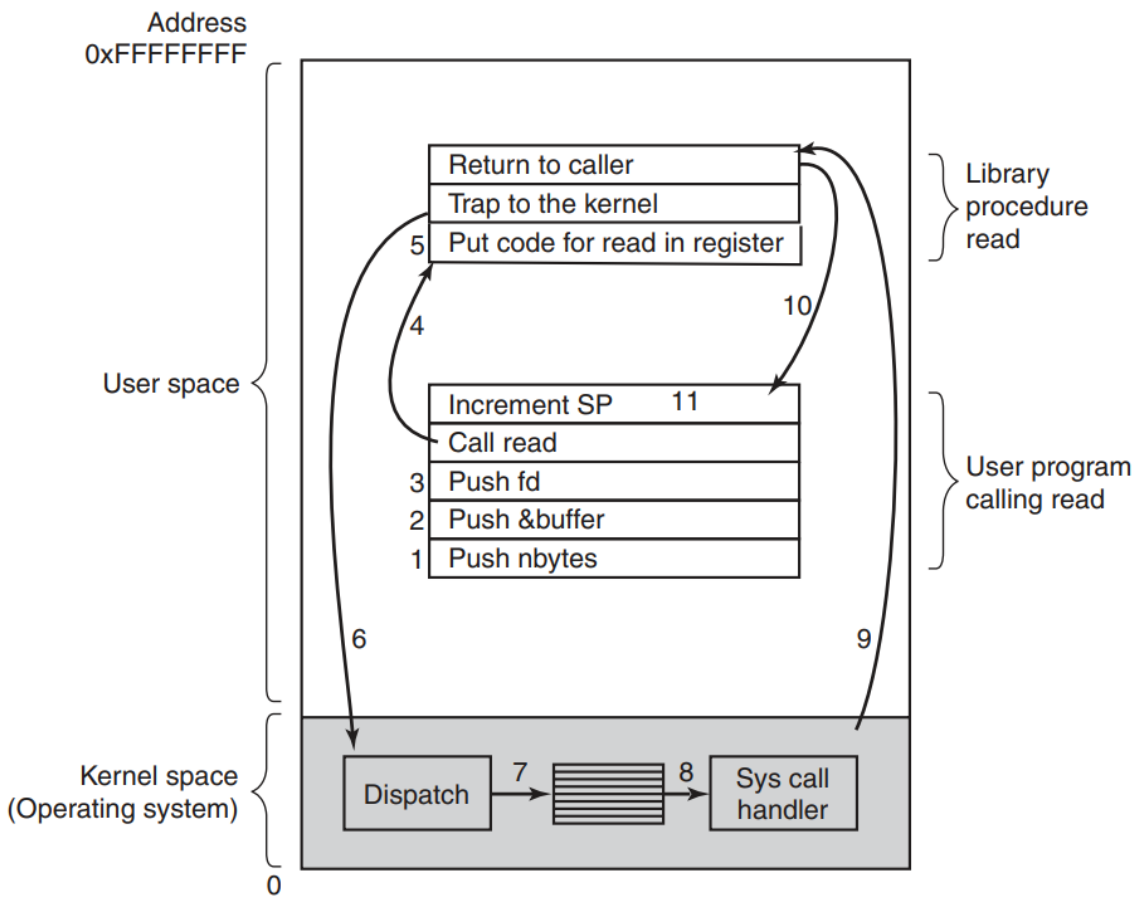


Sistem Çağrıları (System Calls)

- Programlama dilleri, derleyici (compiler) tarafından sağlanan kütüphaneler ile sistem çağrılarına erişmeyi kolaylaştıran arayüz sağlarlar.
- İşletim sistemleri C ve C++ gibi dillerden sistem çağrılarını aktive etmek için bir prosedür kütüphanesi sunarlar.
- Örneğin, C ve C++ ile kod yazarken **printf** komutu sistem çağrısında bulunur.
- Standard C ve C++ kütüphanesine erişen komut sistem çağrısı yapar.



Sistem Çağrıları (System Calls)



- read(fd, buffer, nbytes) sistem çağrısı yapmak için 11 adım.

Sistem Çağrıları (System Calls)

- Sistem çağrıları doğrudan işlemci mimarisine de bağımlıdır.
- Sistem çağrılarının çekirdek tarafındaki gerçekleştirimi işlemci mimarisine göre değişkenlik gösterir.

- Her sistem çağrısının ilişkili bir numarası vardır.
- Bu numaralar da işlemci mimarisine göre değişkenlik gösterir.
- Temel sistem çağrıları tüm mimarilerde bulunmakla birlikte, tüm mimarilerde eşit sayıda sistem çağrısı bulunmaz.

Sistem Çağrıları (System Calls)

- Linux çekirdeğinde X86 mimarisi için yaklaşık **380** civarında sistem çağrısı bulunur.

- Bir işletim sistemi çekirdek hizmetine gereksinim duyulduğunda, örnek olarak Intel 32 bitlik işlemcilerde **(Interrupt) INT 0x80** ile sistem çağrısı oluşturulurken, ARM mimarisinde aynı işlem **(supervisor call) SVC** gibi adlarla anılan özel makina komutları çalıştırılarak ilgili işletim sistemi kesimine sapılır.

Sistem Çağrıları (System Calls)

- Sistem çağrıları aslında oldukça yüksek maliyetli işlemlerdir.
- Her sistem çağrısında process'in o anki durumunun saklanması, çekirdeğin işlemcinin kontrolünü ele alması ve ilgili sistem çağrısı ile çekirdek modunda talep edilen işlemleri gerçekleştirmesi,
- sonra ilgili uygulamanın tekrar yürütme sırası geldiğinde, uygulamanın saklanan durumunun yeniden üretilip process'in kaldığı yerden devamının sağlanması gereklidir.